Accepted at the 2026 vis4climate workshop, which is part of the IEEE VIS 2026 conference. For more information, please visit https://vis4climate.org.

GeoXplain: On-the-Fly Visual Explanations for Weather Foundation Models

Clemens Walter Koprolin, Leonardo Trentini, Benedikt Soja, Mennatallah El-Assady, Christina Humer

ETH Zurich, Zurich, Switzerland

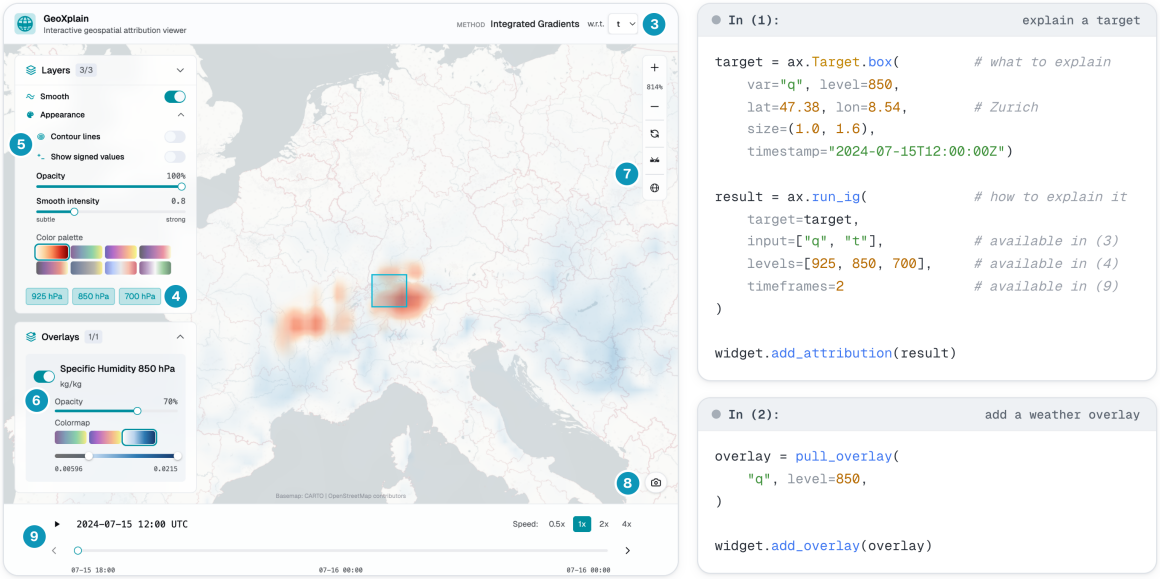


Fig. 1: Example of GeoXplain notebook for explaining a humidity forecast at 850 hPa over Zurich, Switzerland with Integrated Gradients. Attribution data is computed on demand in notebook cell (1), and a humidity forecast overlay is added in cell (2). The left panel shows the resulting interactive view. Users can switch between attribution fields for temperature (t) and specific humidity (q), using control (3). Individual pressure-level explanations can be shown or hidden with control (4), and their visual appearance can be adjusted with the panel (5), for example by switching from filled overlays to contour lines. Panel (6) adjusts the imported overlay, including its color map and opacity. Controls (7) switch between topographic and satellite basemaps and enable a 3D globe view. Control (8) exports the current view as an image, while control (9) provides timeline controls for exploring explanations across forecast times.

Abstract— Weather and climate foundation models produce high-dimensional forecasts whose learned relationships are difficult to inspect with static plots alone. GeoXplain is an interactive Python-based visualization toolkit for exploring geospatial attribution maps across climate variables, atmospheric pressure levels, and forecast time. The toolkit accepts attribution bundles containing attribution grids together with corresponding metadata and renders them in a notebook widget or browser with map and globe modes, linked timelines, pressure-level controls, target annotations, and optional physical-field overlays. We frame GeoXplain as a model-agnostic earth-system visualization toolkit and present the GeoXplain Aurora Adapter as its first computation backend. The adapter computes explanations for the Aurora foundation model, either in a local GPU process, through a GPU listener, or through a SLURM-backed listener, while preserving the same Python call site for analysts. It currently supports gradient saliency, Integrated Gradients, RISE, ViT-CX, multi-frame saliency and Integrated Gradients rollouts, and retrieval of ERA5 overlays. GeoXplain can be installed as a PyPI package with `pip install geoxplain`. The code is open-source and available at https://github.com/clemenskoprolin/geoxplain.

Index Terms—Climate visualization, geospatial visualization, explainable AI, weather foundation models, computational notebooks

1 Introduction

AI-based weather and earth-system models are rapidly becoming practical scientific instruments [4]. These models are attractive for weather- and climate-related workflows because they can produce global forecasts at comparatively low computational cost while remaining competitive with established numerical systems [5, 6, 17]. At the same time, their predictions are distributed over space, atmospheric pressure levels, variables, and lead times, which makes it hard to answer basic interpretability questions: Which input fields influence a humid-air forecast over the Alps? Does the answer change across pressure levels? Does it remain stable over an autoregressive rollout? How should an atmospheric scientist compare the attribution with the underlying meteorological field, such as humidity or temperature?

In practice, scientific model-inspection workflows often combine notebook-based exploration, batch computation, and static plotting, producing intermediate outputs that must be inspected, saved, or reused across stages [3, 14]. This separation slows exploratory explanation work [33], especially for climate and weather data, where the analysis must coordinate high-dimensional spatiotemporal fields across views [8, 20]. Attribution maps are dense global grids, usually meaningful only in relation to a target region, a forecast timestamp, and contextual meteorological fields. Analysts therefore need a tool that behaves like exploratory visual analysis software, but remains close to the

E-mail: {ckoprolin | ltrentini | sojab | melassad | humerc}@ethz.ch

model code and the high-performance computing environment where explanations are produced.

We introduce *GeoXplain*, an interactive toolkit for visual analysis of weather foundation model explanations. GeoXplain is designed around a simple separation: the visualization layer displays attribution results, while model-specific adapters compute them. This paper presents GeoXplain as the main toolkit and the *GeoXplain Aurora Adapter* as a backend that enables on-the-fly computation for Aurora, a leading earth-system foundation model for weather, with applications to air-quality, ocean-wave, and tropical-cyclone prediction [6].

In particular, our contributions are: (1) a visualization workflow for dense geospatial attributions over pressure levels and time; (2) a self-describing result bundle format that decouples the viewer from the computation package; (3) an adaptation of Saliency, Integrated Gradients, RISE, and ViT-CX to Aurora; (4) the Aurora adapter that computes the four explanation methods on demand; and (5) verification evidence for the rendering and numerical behavior of the current implementation. Details on the result bundle format, method runners, and diagnostics are provided in Appendices A–F.

2 Related Work

Weather foundation models. Machine-learning models now match and, on many standard metrics, surpass traditional numerical weather prediction: deterministic systems such as Pangu-Weather, GraphCast, and FourCastNet [5,17,23], and probabilistic ones such as GenCast [25], operate on atmospheric grids and are evaluated with benchmark suites such as WeatherBench 2 [26]. This progress has motivated a shift from task-specific models toward foundation models pretrained across heterogeneous variables and tasks [21, 22]. Microsoft Aurora exemplifies this direction as a flexible foundation model for heterogeneous atmospheric data [6].

Attribution methods. Attribution-based explanation methods help analysts understand model behavior by identifying which input features most influenced a given model output. Gradient-based approaches such as Saliency [28] compute the gradient of a scalar target with respect to the input, while Integrated Gradients [29] integrates such gradients along a path from a baseline input to the observed input. Perturbation-based approaches such as RISE [24] estimate feature influence by randomly masking parts of the input and measuring the resulting changes in model output. Transformer-oriented methods adapt attribution to tokenized internal representations. ViT-CX [32] explains vision-transformer predictions through clustered feature-map masks and their effects on the output. GeoXplain itself is agnostic to the type of explanation used: each model adapter defines how an attribution is computed, and the viewer turns the resulting fields into comparable geospatial visual objects. The adapter introduced in this paper provides Saliency, Integrated Gradients, RISE, and ViT-CX for Aurora.

Such methods have been used for weather-model forecast sensitivity, precipitation and severe-weather prediction, and broader geoscientific model analysis [2, 7, 9, 13, 18]. Recent work has also begun to analyze Aurora with representation analysis and layer-wise relevance propagation [16]. To the best of our knowledge, GeoXplain is the first publicly available system to adapt Saliency, Integrated Gradients, RISE, and ViT-CX to Aurora. This required several Aurora-specific modifications of the methods, which are documented in Appendix A. Since visual plausibility alone is not a reliable validation criterion for attribution maps, our diagnostics include sanity-check practices such as model-parameter randomization [1].

Weather, climate, and ocean visualization. Visualization systems for geophysical forecasting emphasize domain-specific coordinates, time, vertical structure, and uncertainty. Met.3D provides interactive 3D analysis of numerical ensemble weather forecasts, combining meteorological 2D conventions with 3D views and ensemble support [27]. Ovis similarly supports interactive analysis of ocean forecast ensembles with derived quantities recomputed as users explore space, time, and ensemble subsets [12]. GeoXplain shares the concern for preserving geophysical structure, but its primary data are attribution fields for learned models rather than forecast fields or ensemble members.

Interactive ML-explanation visualization. VIS systems have long treated model interpretation as an interactive visual analysis problem, as surveyed by Hohman et al. [10]. ActiVis uses coordinated views for instance- and subset-level exploration of large neural networks [15], while Summit summarizes activations and attributions into scalable interactive representations of learned features and their influence [11]. GeoXplain follows this line of work, but shifts the visual object to georeferenced attribution fields. Its contribution is a domain-specific visualization design and result protocol for comparing explanations across variables, pressure levels, forecast times, and physical overlays.

Notebook-embedded visual analysis. Computational-notebook systems such as B2 [31] and PyGWalker [33] reduce the gap between code-driven analysis and interactive visual exploration by making visual interfaces callable from the analysis environment. GeoXplain follows this on-the-fly pattern; however, attribution computations for foundation models are expensive and may have to be computed on a GPU or SLURM job. This motivates our explicit split between a reusable visualization protocol and model-specific computation adapters, enabling full flexibility in what to compute while maintaining smooth, uninterrupted interactions with the visualizations.

3 GeoXplain

GeoXplain targets weather and climate researchers as well as model developers who build or validate weather models. It supports model inspection by comparing attributions across variables, pressure levels, methods, forecast times, and physical fields.

3.1 Overview

GeoXplain provides two user interfaces: a standalone browser viewer and a Jupyter notebook widget that directly integrates the viewer into the development workflow using `anywidget` [19]. Both interfaces share the same data model, preprocessing, and encoding pipeline, illustrated in Fig. 2. The frontend provides map and globe views, method selection, timeline playback, pressure-level visibility and opacity controls, heatmap or contour rendering, palette selection, target annotations, and optional overlays for physical variables such as specific humidity or temperature (see Fig. 1). Implementation details can be found in Appendix D.

3.2 Design Goals

GeoXplain was developed alongside research that incorporates GNSS-derived Zenith Wet Delay into Aurora for precipitation forecasting done by one of the co-authors [30]. The goals emerged through close collaboration between visualization researchers and this co-author. His workflow and need to validate moisture-related predictions motivated G1–G3, G4 arose from the requirement to support multiple models and methods.

G1: Uninterrupted development workflow. Users should be able to inspect their models without interrupting their development workflow. Many researchers use computational notebooks for their experiments. Integrating the visual analysis within the notebook environment maintains development flow and avoids the common detour through job logs, copied arrays, and static plotting scripts. *G2: Preserve weather-specific context.* Attribution values should remain tied to their meteorological meaning, including variable, forecast time, target location, and vertical pressure level, rather than being shown as unlabeled array dimensions. *G3: Compare explanation and context.* Attribution maps and raw physical-field overlays should be shown in the same coordinate system since meteorological interpretation depends on both. *G4: Isolate model-specific code.* The visualization package should be agnostic to the specific model and attribution method to provide a lightweight, reusable system.

These goals shaped GeoXplain's overall design, including its model-agnostic result bundle format. A GeoXplain result bundle groups attribution grids with the metadata needed to render them, including the explanation method, time frame, target, input variables, and vertical layers. The same format is used by the notebook widget, standalone browser viewer, and remote computation backends. Storage details are provided in Appendix E.

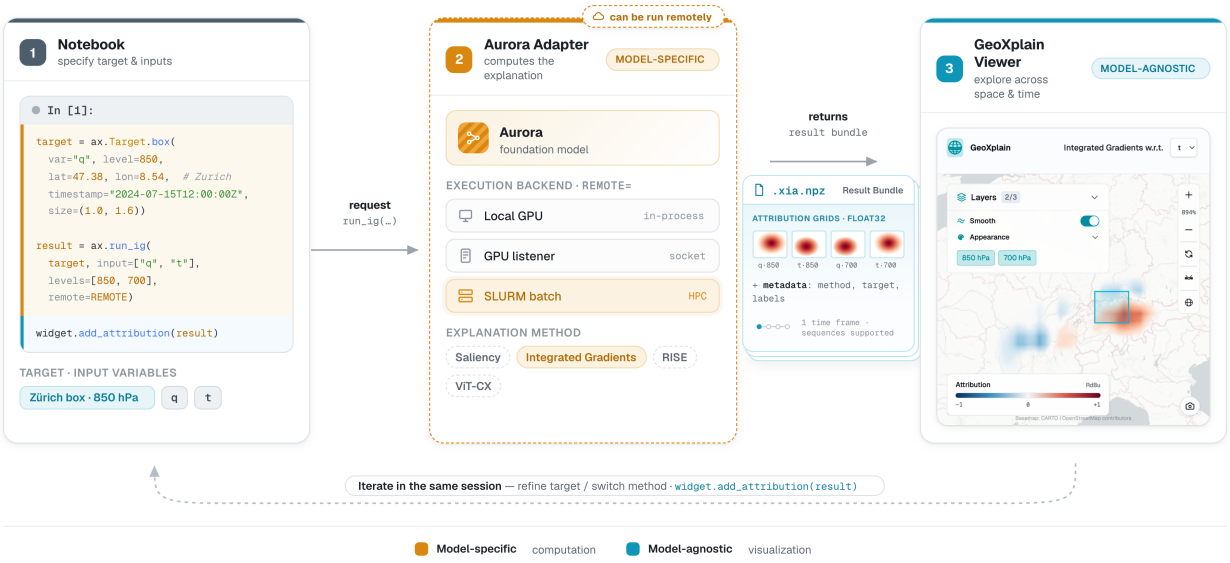


Fig. 2: GeoXplain separates interactive visual analysis from model-specific computation. In the notebook (1), the analyst specifies the target and input variables. The model-specific Aurora adapter (2) turns this target specification into a result bundle containing attribution time frames and metadata. The model-agnostic GeoXplain viewer (3) renders the bundle in a notebook widget or standalone browser view. This example shows how the specific humidity at 850 hPa around Zurich is explained using humidity and temperature at 850 and 700 hPa.

3.3 Visual Representation

GeoXplain renders attribution fields on global latitude–longitude grids as filled heatmaps or contour lines. Heatmaps encode attribution values with continuous color while preserving fine spatial detail [34]. Contours follow familiar meteorological map conventions and keep contextual fields visible when multiple layers would otherwise introduce competing color scales [27]. Signed variables are automatically detected as diverging and displayed with diverging palettes centered at zero by default. Nonnegative variables default to sequential palettes. Users can switch between heatmaps and contour-line depictions and can seamlessly change between a map view and 3D globe view, as shown in Fig. 1.

Physical-field overlays are stored separately from attribution layers. An overlay has its own timestamps, units, and color map, allowing users to compare an explanation map with the physical field that was explained or with another contextual variable. Because overlays use the same timeline as explanations, they can accompany multi-step rollouts and batch time frames.

3.4 Usage Scenario

In GeoXplain, a target defines the forecast output to be explained. It includes the output variable, forecast time, optional vertical level, and spatial region. A point target explains the output at a single coordinate location, whereas a box target first averages the output over a latitude–longitude rectangle and then computes the attribution.

Consider an atmospheric scientist investigating a warm, humid summer case over Zurich, Switzerland, explaining Aurora's subsequent 850 hPa specific-humidity forecast from the 2024-07-15 12:00 UTC input state. In a notebook, the scientist specifies a box target for specific humidity at 850 hPa (≈ 1500 m above sea level) and selects input variables such as temperature, humidity, and geopotential. The call `run_ig(target, input=[...], remote=REMOTE)` submits an Integrated Gradients computation to the specified backend. When the result returns, `w.add_attribution(result)` places each input variable in the method selector and each returned frame on the timeline. The scientist can then switch from the humidity attribution to temperature attribution, compare 850 hPa with higher atmospheric layers, adjust opacity and view contour lines.

The initial IG result already provides a first hypothesis. As shown in the left panel of Fig. 3, the attribution map for the Zurich humidity target assigns positive contribution to 850 hPa temperature southeast of Zurich and negative contribution to a region near Lyon. This suggests that Aurora's humidity prediction is sensitive not only to local humidity, but also to nearby thermal structure and upstream conditions.

The same session can then be extended. To test whether this pattern is method-specific, the scientist requests RISE for the same target. RISE highlights similar regions, which increases confidence that the observed patches are not merely an artifact of the IG baseline path.

To inspect the meteorological context, the scientist adds physical overlays. With `pull_overlay("q", level=850, remote=REMOTE)`, the 850 hPa humidity field is retrieved from the remote backend. Metadata such as the default color map are inferred from the request, and the adapter provides the previously used timestamps automatically. After calling `w.add_overlay(overlay)`, the scientist adds the field to the running viewer and repeats the same step for temperature. The attributed regions, especially the one to the south, coincide with slightly warmer 850 hPa air, suggesting that the model may use local thermal structure as part of the humidity forecast. However, this visual agreement alone is not sufficient evidence for a physical mechanism.

To see whether the same regions remain influential over later forecast steps, the scientist requests an autoregressive rollout with `run_rollout(..., method="ig", timeframes=5)`, which iteratively uses each forecast state as input for the subsequent forecast step and computes each IG attribution with respect to the original input state. GeoXplain displays the returned attribution frames as a playable sequence. Across longer lead times, the initial patches weaken, while stronger attribution appears farther upstream over western France and northern Spain, as shown in the right panel of Fig. 3.

The scientist then adds eastward and northward wind-component overlays, using custom color maps with `w.add_overlay(overlay, colormap=...)` to separate them visually. The wind fields show persistent northeastward flow from northern Spain and western France toward the Zurich region. This makes the attribution pattern meteorologically plausible: the model may be responding to upstream air-mass transport rather than only to local humidity or temperature anomalies.

The scientist then uses the export-as-image feature to export the current view for documentation and calls `result.save(...)` to store the explanations as a result bundle that can be shared with colleagues.

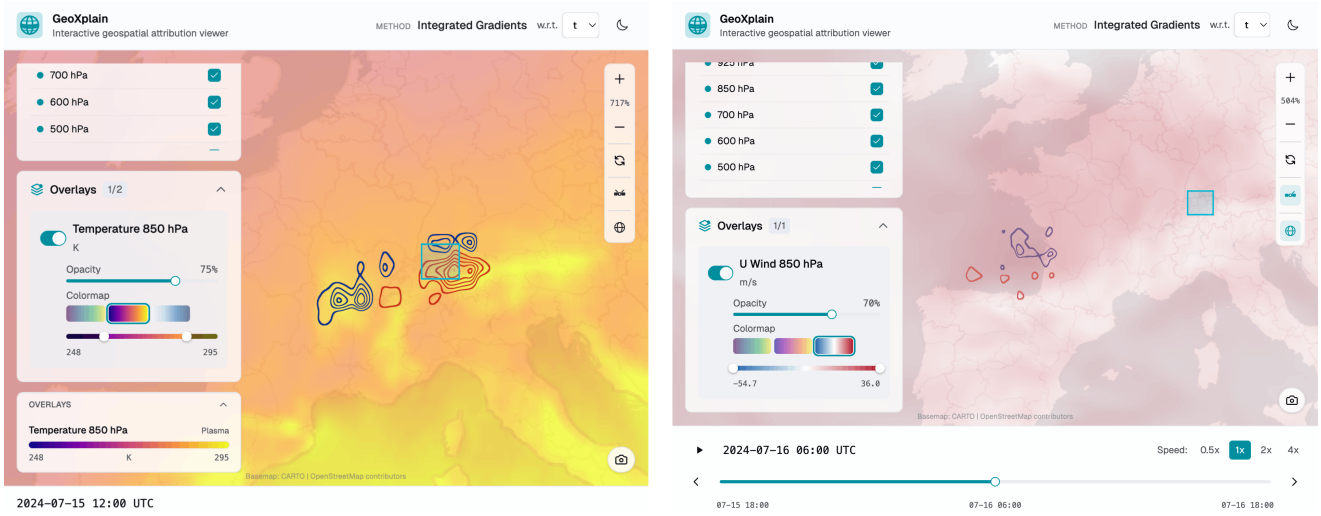


Fig. 3: The left image shows IG attributions for the 850 hPa humidity forecast over Zurich with an 850 hPa temperature overlay. The right image shows the second autoregressive rollout step with the 850 hPa horizontal wind field. Positive wind values indicate an eastward flow. In both panels, attribution is shown by contour lines, with red indicating positive and blue indicating negative attribution.

4 On-the-fly Computation

GeoXplain uses adapters as computation backends between model code and the viewer. An adapter translates a target and explanation request into GeoXplain's standardized result bundle, keeping the visualization package independent of any particular model implementation, data loader, attribution method, or execution environment. In practice, adapters resolve targets, prepare model inputs, run the requested explanation or overlay computation, and package attribution grids and metadata in the common format.

4.1 Aurora Adapter

The Aurora adapter implements this backend pattern for Aurora and WeatherBench-style case data. A target defines the output variable, pressure level, timestamp, and either a point or a latitude-longitude box (Fig. 2, (1)). The adapter loads the matching WeatherBench-style case data, builds an Aurora batch, constructs a differentiable scalar target, runs the selected explanation method, and returns attribution maps for the requested input variables, as shown in Fig. 2, (2). For a single timestamp, the result contains one frame. For batch time frames or rollouts, it contains multiple frames that GeoXplain places on the timeline.

The same Python call can be executed locally, through a GPU listener, or through a SLURM-backed listener, corresponding to the execution backends in Fig. 2, (2). Remote requests use status polling with progress and log tails, and return a result bundle when complete. This addresses a practical mismatch between interactive visual analysis and HPC (high-performance computing) execution: the analyst wants to make small target or method changes in a notebook, while the model may need a GPU node and containerized runtime. The adapter hides these constraints behind the `remote` argument, and the returned result object is identical in all execution modes.

4.2 Data Flow

The adapter computes the results for each attribution method and returns the outputs in the standardized result bundle format. For each request, the adapter prepares the Aurora batch, resolves the target to a scalar output, and masks the requested fields for attribution. It then converts the returned arrays into GeoXplain frames including the variable, pressure level, color map, and runtime metadata. Batch time frames repeat this process over consecutive timestamps and merge the frames into one bundle. Rollout mode instead backpropagates through an autoregressive sequence, so a single target can ask which initial inputs mattered for later lead times. Further implementation details are provided in Appendix F.

4.3 Adapting Other Models

GeoXplain's model-specific interface is intentionally narrow. A new model backend must provide (1) a target specification that resolves to a scalar model output, (2) a case loader and batch builder, (3) method runners that produce layer grids, and (4) a serializer that satisfies the result bundle format. Nothing in the GeoXplain viewer requires Aurora-specific imports. This separation is important because the same visualization questions arise for regional models, climate emulators, downscaling models, and foundation models. For example, regional precipitation or climate-emulator adapters could reuse the viewer while replacing only the target resolver, data loader, model wrapper, and layer labels. The visual comparison tasks remain the same: select a target, inspect the attribution field, compare across variables and time, and check the explanation against physically meaningful overlays.

5 Verification

The viewer is tested with visual parity checks that render synthetic ramps, blobs, and checkerboards, capture the output, and compare screenshots against Matplotlib references for color maps, opacity, and smoothing. The adapter has unit tests for API routing, batch/time-frame expansion, remote polling, overlay serialization, and login-node overlay execution. Numerical diagnostics were run on four GH200 GPUs with the public Aurora 6 h model, using multiple target cases for randomization and method-specific checks. Integrated Gradients showed sub-percent to about 1% completeness residuals in most cases, and full-model randomization removed learned attribution structure. We treat these results as necessary sanity checks rather than a complete diagnostic analysis. Details are provided in Appendices B and C.

6 Conclusion and Future Work

GeoXplain connects interactive geospatial visualization with on-the-fly explanation computation for weather and climate foundation models. Its Aurora adapter demonstrates the proposed result protocol, which can be adapted to other models and workloads. The central contribution is an accessible way to inspect, compare, and discuss model explanations in their geophysical context. Costly ViT-CX computations and long rollouts can lead to substantial waiting times. GeoXplain separates computation from interaction to reduce workflow disruption. Although design choices were developed in close collaboration with a domain expert, future plans include evaluating GeoXplain with a larger group of end users. Finally, the current protocol emphasizes dense regular grids optimized for Aurora and WeatherBench-style data. Supporting unstructured meshes, station observations, ensembles, uncertainty fields, and multi-resolution regional nests will require protocol extensions.

Software Availability

GeoXplain is released as two open-source Python packages, `geoxplain` and `geoxplain-aurora-adapter`, installable via PyPI. Source code, documentation, and runnable examples are available at the project repository `https://github.com/clemenskoprolin/geoxplain`.

Acknowledgments

This work was supported by the ETH AI Center postdoctoral fellowship awarded to Christina Humer and the International Geneva AI Innovation Programme (IGAIP) with funding from Swiss Foreign Ministry. The contributions of Leonardo Trentini were supported by the Swiss National Science Foundation (SNSF) under grant number 225851. This work was further supported as part of the Swiss AI Initiative through a grant from the ETH Domain, with computational resources provided by the Swiss National Supercomputing Centre (CSCS) under the Alps infrastructure (project IDs a122 and a0196).

References

[1] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim. Sanity checks for saliency maps. In *Proc. NeurIPS*, pp. 9505–9515, 2018. 2

[2] J. Baño-Medina, A. Sengupta, J. D. Doyle, C. A. Reynolds, D. Watson-Parris, and L. Delle Monache. Are AI weather models learning atmospheric physics? a sensitivity analysis of cyclone Xynthia. *npj Climate and Atmospheric Science*, 8(1):92, 2025. doi: 10.1038/s41612-025-00949-6 2

[3] M. Beg, J. Taka, T. Kluyver, A. Konovalov, M. Ragan-Kelley, N. M. Thiéry et al. Using Jupyter for reproducible scientific workflows. *Computing in Science & Engineering*, 23(2):36–46, 2021. doi: 10.1109/MCSE.2021.3052101 1

[4] Z. Ben Bouallègue, M. C. A. Clare, L. Magnusson, E. Gascon, M. Maier-Gerber, M. Janousek et al. The rise of data-driven weather forecasting: A first statistical assessment of machine learning-based weather forecasts in an operational-like context. *Bulletin of the American Meteorological Society*, 105(6):E864–E883, 2024. doi: 10.1175/BAMS-D-23-0162.1 1

[5] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian. Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, 619:533–538, 2023. doi: 10.1038/s41586-023-06185-3 1, 2

[6] C. Bodnar, W. P. Bruinsma, A. Lucic, M. Stanley, A. Allen, J. Brandstetter et al. A foundation model for the Earth system. *Nature*, 641:1180–1187, 2025. doi: 10.1038/s41586-025-09005-y 1, 2

[7] P. L. Bommer, M. Kretschmer, A. Hedström, D. Bareeva, and M. M.-C. Höhne. Finding the right XAI method—a guide for the evaluation and ranking of explainable AI methods in climate science. *Artificial Intelligence for the Earth Systems*, 3(3):e230074, 2024. doi: 10.1175/AIES-D-23-0074.1 2

[8] A. Diehl, L. Pelorosso, C. Delrieux, C. Saulo, J. Ruiz, M. E. Gröller et al. Visual analysis of spatio-temporal data: Applications in weather forecasting. *Computer Graphics Forum*, 34(3):381–390, 2015. doi: 10.1111/cgf.12650 1

[9] L. Espeholt, S. Agrawal, C. Sønderby, M. Kumar, J. Heek, C. Bromberg et al. Deep learning for twelve hour precipitation forecasts. *Nature Communications*, 13(1):5145, 2022. doi: 10.1038/s41467-022-32483-x 2

[10] F. Hohman, M. Kahng, R. Pienta, and D. H. Chau. Visual analytics in deep learning: An interrogative survey for the next frontiers. *IEEE Transactions on Visualization and Computer Graphics*, 25(8):2674–2693, 2019. doi: 10.1109/TVCG.2018.2843369 2

[11] F. Hohman, H. Park, C. Robinson, and D. H. Chau. Summit: Scaling deep learning interpretability by visualizing activation and attribution summarizations. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1096–1106, 2020. doi: 10.1109/TVCG.2019.2934659 2

[12] T. Höllt, A. Magdy, P. Zhan, G. Chen, G. Gopalakrishnan, I. Hoteit et al. Ovis: A framework for visual analysis of ocean forecast ensembles. *IEEE Transactions on Visualization and Computer Graphics*, 20(8):1114–1126, 2014. doi: 10.1109/TVCG.2014.2307892 2

[13] Z. Hua, R. A. Sobash, D. J. Gagne, II, Y. Sha, and A. Anderson-Frey. Improving medium-range severe weather prediction through transformer postprocessing of AI weather forecasts. *Artificial Intelligence for the Earth Systems*, 5(1):250045, 2026. doi: 10.1175/AIES-D-25-0045.1 2

[14] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55 1

[15] M. Kahng, P. Y. Andrews, A. Kalro, and D. H. Chau. ActiVis: Visual exploration of industry-scale deep neural network models. *IEEE Transactions on Visualization and Computer Graphics*, 24(1):88–97, 2018. doi: 10.1109/TVCG.2017.2744718 2

[16] E. Kasteleyn and A. Lucic. Does Aurora encode atmospheric structure? latent regime analysis and attribution. In *ICLR 2026 Workshop on Foundation Models for Science*, 2026. Poster. 2

[17] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirnsberger, M. Fortunato, F. Alet et al. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023. doi: 10.1126/science.adi2336 1, 2

[18] A. Mamalakis, E. A. Barnes, and I. Ebert-Uphoff. Investigating the fidelity of explainable artificial intelligence methods for applications of convolutional neural networks in geoscience. *Artificial Intelligence for the Earth Systems*, 1(4):e220012, 2022. doi: 10.1175/AIES-D-22-0012.1 2

[19] T. Manz, N. Abdennur, and N. Gehlenborg. anywidget: reusable widgets for interactive analysis and visualization in computational notebooks. *Journal of Open Source Software*, 9(102):6939, 2024. doi: 10.21105/joss.06939 2

[20] A.-A.-R. Nayeem, D. Han, H. Lee, D. Kim, D. Feldman, W. J. Tolone et al. A survey on exploratory spatiotemporal visual analytics approaches for climate science, 2024. doi: 10.48550/arXiv.2407.21199 1

[21] T. Nguyen, J. Brandstetter, A. Kapoor, J. K. Gupta, and A. Grover. ClimaX: A foundation model for weather and climate. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. 2

[22] F. Ozdemir, Y. Cheng, S. Mohebi, F. Lehmann, S. Adamov, Z. Zhang et al. Earth system foundation model (esfm): A unified framework for heterogeneous data integration and forecasting, 2026. 2

[23] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani et al. FourCastNet: A global data-driven high-resolution weather model using adaptive Fourier neural operators, 2022. 2

[24] V. Petsiuk, A. Das, and K. Saenko. RISE: Randomized input sampling for explanation of black-box models. In *Proc. BMVC*, 2018. 2

[25] I. Price, A. Sanchez-Gonzalez, F. Alet, T. R. Andersson, A. El-Kadi, D. Masters et al. Probabilistic weather forecasting with machine learning. *Nature*, 637(8044):84–90, 2025. doi: 10.1038/s41586-024-08252-9 2

[26] S. Rasp, S. Hoyer, A. Merose, I. Langmore, P. Battaglia, T. Russell et al. WeatherBench 2: A benchmark for the next generation of data-driven global weather models. *Journal of Advances in Modeling Earth Systems*, 16(6):e2023MS004019, 2024. doi: 10.1029/2023MS004019 2

[27] M. Rautenhaus, M. Kern, A. Schäfler, and R. Westermann. Three-dimensional visualization of ensemble weather forecasts – part 1: The visualization tool Met.3D (version 1.0). *Geoscientific Model Development*, 8(7):2329–2353, 2015. doi: 10.5194/gmd-8-2329-2015 2, 3

[28] K. Simonyan, A. Vedaldi, and A. Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *Workshop at International Conference on Learning Representations*, 2014. 2

[29] M. Sundararajan, A. Taly, and Q. Yan. Axiomatic attribution for deep networks. In *Proc. ICML*, pp. 3319–3328, 2017. 2

[30] L. Trentini, F. Lehmann, L. Crocetti, and B. Soja. Integrating gnss-derived zenith wet delay into a weather foundation model improves precipitation forecasting, 2026. 2

[31] Y. Wu, J. M. Hellerstein, and A. Satyanarayan. B2: Bridging code and interactive visualization in computational notebooks. In *Proc. ACM Symposium on User Interface Software and Technology*, pp. 152–165, 2020. doi: 10.1145/3379337.3415851 2

[32] W. Xie, X.-H. Li, C. C. Cao, and N. L. Zhang. ViT-CX: Causal explanation of vision transformers. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, pp. 1569–1577. International Joint Conferences on Artificial Intelligence Organization, 2023. doi: 10.24963/ijcai.2023/174 2

[33] Y. Yu, L. Shen, F. Long, H. Qu, and H. Chen. PyGWalker: On-the-fly assistant for exploratory visual data analysis. In *Proc. IEEE VIS*, pp. 6–10, 2024. doi: 10.1109/VIS55277.2024.00009 1, 2

[34] Y. Zhao, M. Li, and M. Berger. Graphical perception of saliency-based model explanations. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, art. no. 821, 15 pages. Association for Computing Machinery, New York, NY, USA, 2023. doi: 10.1145/3544548.3581320 3

A METHOD RUNNER ADAPTATIONS

All method runners explain the same scalar target. Let X denote the Aurora input case and let

$$S(X) = \tau(F(X))$$

be the scalar obtained by applying Aurora F and resolving the user target τ: either a point value or an arithmetic grid-cell mean over a target box, at a selected pressure level for atmospheric variables.

Requested input variables are denoted by $v \in \mathcal{V}$. Atmospheric fields include two input time steps, pressure levels, and the horizontal grid, while surface fields omit the pressure-level dimension. We write t_1 for the second input time step, X' for a smoothed-baseline case, and $\delta = 10^{-8}$ for guarded divisions.

The exported attribution is always with respect to the second input time step, t_1, because the requested timestamp is interpreted as Aurora's current input state. Preliminary testing showed that this choice produced more interpretable attribution maps, although future work could also consider attributions with respect to the earlier input state t_0.

Saliency. For each requested variable, saliency is the gradient of the scalar target with respect to the selected input field,

$$A_v^{\mathrm{sal}} = \frac{\partial S(X)}{\partial X_{v,t_1}}.$$

All requested variables are marked as differentiable inputs, and their gradients are computed in a single backward pass. Atmospheric gradients are then split into pressure-level maps, while surface gradients are stored under the reserved `sfc` layer.

Integrated Gradients. Integrated Gradients attributes the target by accumulating gradients along a path from a smoothed baseline X' to the actual input X. In this adapter, X' is obtained by Gaussian smoothing the input fields in latitude and longitude, using reflected latitude boundaries and wrapped longitude boundaries.

For each requested variable,

$$A_v^{\mathrm{IG}} = (X_{v,t_1} - X'_{v,t_1}) \frac{1}{n} \sum_{k=1}^{n} \frac{\partial S(X^{(k)})}{\partial X_{v,t_1}^{(k)}}, \qquad X^{(k)} = X' + \frac{k-0.5}{n}(X - X'),$$

where n is the number of integration steps. The points $X^{(k)}$ are midpoint samples on the straight path from X' to X.

For selected variables, the IG path interpolates the selected variable fields, including both input time steps, and the t_1 attribution is exported. Unselected fields remain fixed at their original input values. The target resolver and export convention are the same as for saliency. Future work could investigate alternative baselines or other integration paths.

RISE. RISE estimates attribution by perturbing one requested input variable at a time and measuring how the scalar target changes. In this adapter, perturbations are applied only to the second input time step t_1, while t_0 is kept fixed.

For a random horizontal keep mask $M^{(k)}$, the selected field is blended with its smoothed baseline,

$$\tilde{X}_{v,t_1}^{(k)} = M^{(k)} \odot X_{v,t_1} + (1 - M^{(k)}) \odot X'_{v,t_1},$$

where $\odot$ denotes element-wise multiplication. The modified case $\tilde{X}_v^{(k)}$ is otherwise identical to X. For atmospheric variables, the same horizontal mask is applied at all pressure levels, so the perturbation is column-wise. The masks are generated from low-resolution Bernoulli grids, then bilinearly upsampled into continuous masks in $[0,1]$ and shifted circularly in longitude to avoid dateline artifacts.

Rather than using the original classification-style RISE normalization, the adapter uses a centered regression estimator,

$$A_{v,i}^{\mathrm{RISE}} = \begin{cases} \dfrac{\sum_{k=1}^{N} (S(\tilde{X}_v^{(k)}) - \bar{S})(M_i^{(k)} - \bar{M}_i)}{\sum_{k=1}^{N} (M_i^{(k)} - \bar{M}_i)^2}, & \mathrm{var}(M_i) > \delta, \\ 0, & \text{otherwise}, \end{cases}$$

where i indexes a horizontal grid cell, N is the number of masks,

$$\bar{S} = \frac{1}{N} \sum_{k=1}^{N} S(\tilde{X}_v^{(k)}), \qquad \bar{M}_i = \frac{1}{N} \sum_{k=1}^{N} M_i^{(k)}.$$

This estimator removes the constant forecast offset and assigns positive values to regions whose visibility is associated with a higher scalar target. Because atmospheric masks are applied column-wise, atmospheric variables are exported as one collapsed horizontal attribution map per variable.

ViT-CX. ViT-CX requires an Aurora-specific adaptation because Aurora's encoder is hierarchical and does not expose a single fixed image-patch grid. The runner therefore hooks a user-selected encoder stage, `hook_stage`. Each stage has its own horizontal token resolution, feature dimension, and compressed vertical token groups.

For stage 2, the recorded activation is reshaped as

$$Z \in \mathbb{R}^{G \times H_e \times W_e \times D}, \qquad \bar{Z}_{ij:} = \frac{1}{G} \sum_{g=1}^{G} Z_{gij:},$$

where $G = 4$ denotes the vertical token groups, $(H_e, W_e) = (45, 90)$ is the encoder-grid resolution, and D is the feature dimension. The vertical groups are averaged so that each horizontal encoder cell is represented by one feature vector $\bar{Z}_{ij:}$.

Perturbation regions are then obtained by clustering the normalized spatial token vectors $\bar{Z}_{ij:}$. This differs from the original ViT-CX formulation, which constructs masks from embedding-channel feature maps. The clustering produces masks $C_1, \ldots, C_K$ on the encoder grid, with K set by `n_clusters`. Each mask represents a spatial region. For atmospheric variables, the corresponding horizontal region is applied to all pressure levels.

Each cluster mask is upsampled to the full input grid and used as an occlusion region. Let $X_v^{(k,\mathrm{occ})}$ denote the modified case in which the selected t_1 field of variable v is replaced by its smoothed baseline inside cluster C_k. The cluster score is

$$s_k = S(X) - S\left(X_v^{(k,\mathrm{occ})}\right).$$

Positive scores therefore indicate regions whose removal decreases the target.

The final attribution map is formed by assigning each cluster score back to its spatial support,

$$A_v^{\mathrm{CX}}(i) = \begin{cases} \dfrac{\sum_{k=1}^{K} s_k \mathbf{1}\{i \in C_k\}}{\sum_{k=1}^{K} \mathbf{1}\{i \in C_k\}}, & \sum_{k=1}^{K} \mathbf{1}\{i \in C_k\} > \delta, \\ 0, & \text{otherwise}, \end{cases}$$

where i indexes a horizontal grid cell. The resulting encoder-grid map is upsampled to the full grid and, optionally, smoothed. As in RISE, atmospheric variables are perturbed column-wise and therefore produce one collapsed horizontal attribution map per input variable.

Rollout. Rollout changes the forecast horizon while keeping the attribution input fixed. For rollout frame r, Aurora is applied autoregressively for $r+1$ six-hour steps and the scalar target is evaluated on the final prediction,

$$S_r(X) = \tau(F^{r+1}(X)).$$

Saliency or Integrated Gradients is then computed for S_r with respect to the original input case X, not with respect to the intermediate predicted states. Thus each rollout frame asks how the initial input state contributes to the selected target at a later lead time.

The returned result bundle stores one frame per lead time, with metadata for rollout index, lead hours, runtime, and target score. RISE and ViT-CX are excluded from rollout because their perturbation loops would require many full autoregressive forwards per variable. Future work could investigate this.

B XAI METHODS VERIFICATION

Diagnostic setup. The following checks are intended as implementation diagnostics, not as an evaluation of forecast skill or as a comparison against meteorological baselines. They used the public Aurora 6 h model and were run across four GH200 GPUs. The shared diagnostic parameters are summarized in Table 1. The randomization checks used three target cases: an Alpine land q@850 box, a North Atlantic t@500 box, and a tropical Pacific q@700 box. The broader Integrated Gradients completeness suite also included additional seasonal and variable-set variants. Table 2 lists the target geometry and attribution variables for all reported cases. These checks are sanity tests: passing them is necessary for trustworthy attribution, but does not by itself establish scientific validity. The diagnostic scripts are available for reproducibility at https://github.com/clemenskoprolin/geoxplain-aurora-adapter/tree/master/diagnostics.

Table 1: Main parameters of the numerical diagnostics.

Item	Value
Model	Public Aurora 6 h pretrained checkpoint (`microsoft/aurora`)
IG baseline	Gaussian-smoothed input, $\sigma = 2.5°$
IG step counts	$4, 8, 16, 32, 64$ midpoint samples
RISE parameters	An 18×36 low-resolution mask grid
Randomization seeds	$11, 23, 37, 51, 67$ for full randomization
Cascade seeds	$11, 23$, output-to-input cumulative randomization
RISE convergence seeds	Mask seeds: clean A = 42, clean B = 1234
RISE convergence checkpoints	$32, 64, 128, 256, 512, 1024$ masks, same 18×36 grid
ViT-CX parameters	Encoder stage 2, with token grid $4 \times 45 \times 90$

Integrated Gradients completeness. For Integrated Gradients, the main numerical diagnostic was completeness. The baseline X' was constructed by Gaussian smoothing the input fields in latitude and longitude, and the path integral was approximated with midpoint samples. We compared the summed attribution with the target-score difference $S(X) - S(X')$. The integration path interpolates both input time steps, so the sum runs over t_0 and t_1 jointly. The per-variable t_1 slice used for visualization is only a part of this total, and is not expected to satisfy completeness on its own. Table 3 reports the 64-step results for the full completeness suite: seasonal Alpine q@850 boxes, single-, three-, and five-variable attribution sets, and three auxiliary non-q@850 targets. For the Alpine q@850 box targets, residuals were sub-percent except for the winter q-only run, which reached 1.02%. The auxiliary cases ranged from 0.02% for tropical q@700 to 2.97% for the point z@500 target, where the absolute gap was still small relative to the attribution energy. Thus, for the q@850 box setting, IG completeness can be summarized as sub-percent to about 1%, with the wider diagnostic suite reaching about 3% in the point-target case.

Table 3: Integrated Gradients completeness at 64 midpoint samples. Gap and energy are relative percentages.

Case	Target	Δ	ΣIG	gap	energy
`alps_w`	q@850	-2.71×10^{-4}	-2.72×10^{-4}	0.31	0.018
`alps_p`	q@850	-1.25×10^{-4}	-1.26×10^{-4}	0.58	0.009
`alps_s`	q@850	4.30×10^{-4}	4.30×10^{-4}	0.05	0.002
`alps_a`	q@850	2.48×10^{-4}	2.47×10^{-4}	0.49	0.014
`alps_w_q`	q@850	-2.76×10^{-4}	-2.79×10^{-4}	1.02	0.181
`alps_w_all`	q@850	-3.14×10^{-4}	-3.15×10^{-4}	0.35	0.015
`atl_s`	t@500	5.19×10^{-1}	5.21×10^{-1}	0.39	0.013
`trop_s`	q@700	-1.05×10^{-3}	-1.05×10^{-3}	0.02	0.005
`pt_s`	z@500	7.94	7.71	2.97	0.003

Parameter randomization. The parameter-randomization sanity check followed Adebayo et al.: attributions from the trained model were compared with attributions after progressively randomizing model components. First, cascading randomization progressively randomized parameter groups from the Perceiver decoder back toward the input encoder. Attribution similarity and spatial energy decreased as more of the network was randomized, and all methods reached full collapse once the encoder and input tokenizer were included, with energy ratios approaching zero and 100% collapse at the final stage. Second, Table 4 reports the full-model randomization endpoint over the three target cases and five random seeds. IG, RISE, and ViT-CX collapsed to near-constant attribution fields, while saliency retained only negligible spatial energy. The mean energy ratio was about 6×10^{-5} for IG and RISE, 2×10^{-4} for ViT-CX, and 7×10^{-4} for saliency. Correlation-based similarities were therefore at or below the calibrated spatial-noise baseline. This supports the sanity requirement that the maps depend on the learned Aurora parameters rather than only on fixed preprocessing, target geometry, or visualization choices.

Table 4: Full-model randomization over three cases and five seeds.

Method	Runs	Collapse	Pearson	Energy ratio
Saliency	15	10/15	0.009 ± 0.021	$(6.95 \pm 7.6)\times10^{-4}$
IG	15	15/15	–	$(5.85 \pm 3.9)\times10^{-5}$
RISE	15	15/15	–	$(6.34 \pm 4.7)\times10^{-5}$
ViT-CX	15	15/15	–	$(1.93 \pm 1.7)\times10^{-4}$

RISE Monte Carlo convergence. RISE was additionally checked for Monte Carlo convergence using the mask counts listed in Table 1. With 32 to 1024 masks, the self-consistency of the estimate increased monotonically on the tested cases, with mean self-correlation rising from about 0.30 to 1.00. Agreement between independent mask draws also improved, from about 0.06 to 0.68 mean Pearson correlation. This confirms the expected variance reduction with more perturbation samples, while also showing why RISE is more expensive and noisier than the gradient methods at low mask counts.

Summary and scope. Taken together, these diagnostics indicate that IG has the expected completeness behavior for the tested targets, RISE shows the expected sample-count dependence, and full-model randomization removes the learned structure from the attribution maps. However, the suite only covers nine case variants, and most methods were tested with only one parameter setting. We therefore treat these results as preliminary implementation evidence and leave broader tests across variables, targets, locations, method settings, and input-perturbation robustness to future work.

C VIEWER VERIFICATION

Verification setup. The viewer is checked at two complementary levels. Protocol tests exercise the data handoff from Python into the standalone browser viewer, notebook widget, and remote backends. These tests focus on whether targets, overlays, timelines, input variables and other metadata are serialized consistently. A separate visual parity suite then checks whether the browser rendering of those payloads has the intended appearance.

For the visual checks, synthetic attribution fields are created and prepared for viewing. The suite renders independent Matplotlib references, starts the viewer, and captures the map output. Each viewer capture is generated from a fixed launch state, so the map camera, map style, visible layer, and smoothing options are controlled. The run writes reference images, viewer screenshots, and side-by-side comparison panels for inspection.

Test cases. Each synthetic test isolates a visible property of the viewer. West-to-east ramps test the diverging and sequential color maps. Gaussian blobs test how weak and strong attribution values fade or saturate under the opacity rule. Alternating positive and negative stripes test the near-zero transparent band in the signed display. Checkerboards and single-cell spikes test imported-grid smoothing with smoothing disabled and enabled.

The Matplotlib reference renderer follows the same normalization and opacity conventions as the viewer, while remaining independent of the browser implementation. Some differences are expected: the

Table 2: Diagnostic cases and target geometry. Longitudes use the ERA5 0°–360° convention. Box sizes are latitude by longitude in degrees.

Case	Init	Target	Mode	Center/point (ϕ, λ)	Size	Attrib. vars
`alps_w`	2024-01-15 00Z	q@850	box	(46.25, 8.75)	(1.5, 2.5)	t, q, z
`alps_p`	2024-04-10 12Z	q@850	box	(46.25, 8.75)	(1.5, 2.5)	t, q, z
`alps_s`	2024-07-15 00Z	q@850	box	(46.25, 8.75)	(1.5, 2.5)	t, q, z
`alps_a`	2024-10-15 12Z	q@850	box	(46.25, 8.75)	(1.5, 2.5)	t, q, z
`alps_w_q`	2024-01-15 00Z	q@850	box	(46.25, 8.75)	(1.5, 2.5)	q
`alps_w_all`	2024-01-15 00Z	q@850	box	(46.25, 8.75)	(1.5, 2.5)	t, q, z, u, v
`atl_s`	2024-07-15 00Z	t@500	box	(50.00, 330.00)	(3.0, 4.0)	t, q, z
`trop_s`	2024-07-15 00Z	q@700	box	(5.00, 200.00)	(4.0, 5.0)	q
`pt_s`	2024-07-15 00Z	z@500	point	(40.00, 280.00)	–	t, q, z, u, v

reference image uses a neutral background, whereas the viewer is drawn over MapLibre topographic tiles. The reference is shown on a longitude–latitude grid, whereas the viewer uses Web Mercator.

Results. The protocol tests and visual parity cases behave as expected. The exported payloads are accepted consistently by the viewer entry points, and the browser captures reproduce the intended color maps, transparency behavior, sign handling, and smoothing choices. The generated comparison panels show agreement with the Matplotlib references apart from the documented background, projection, and cropping differences. Fig. 4 shows one representative comparison for the sequential opacity-blob case. The verification suite is provided at `https://github.com/clemenskoprolin/geoxplain/tree/main/tests/visual_test` for reproducibility.

D GeoXplain Implementation

The `geoxplain` package keeps the visualization layer independent of any computation backend. `add_attribution(...)` accepts arrays, file paths, level mappings, or attribution bundles, and `add_overlay(...)` accepts arrays, NetCDF variables, or overlay bundles, so any producer matching these interfaces can drive the viewer. During ingestion, each array is cast to `float32`, validated as a two-dimensional grid, then stored by method, frame, and layer.

From this store, one versioned viewer payload is built. Each grid is normalized, quantized to an unsigned-byte texture, and base64 encoded: signed attribution maps are scaled symmetrically around zero so positive and negative contributions remain distinguishable after quantization, while sequential overlays use a global minimum and maximum across all frames. Every texture carries target, timestamp, color map, layer label, and content-hash metadata, where the hash lets the frontend detect unchanged data and skip redundant reloading. This payload is then synchronized either with the widget or with the browser. A common `AttributionViewer` then decodes the payload into typed dense grids and uploads the selected attribution and overlay frames as WebGL textures. In map mode these textures are drawn through MapLibre custom layers, while globe mode renders them with Three.js on the globe geometry. In both cases shader programs sample the encoded grids, apply color maps, opacity, smoothing, contours, and frame blending.

E Result, Target, and Overlay Formats

The boundary between `geoxplain` and a computation backend is a data format rather than a shared Aurora dependency. The viewer accepts objects that satisfy the attribution and overlay interfaces, while the Aurora adapter is the producer of those objects.

The target specification defines the scalar forecast output to explain: output variable, optional pressure level, timestamp, and spatial mode. Point targets resolve to the nearest grid cell. Box targets store a center and full latitude–longitude extent and are evaluated as the mean over that box.

Attribution bundles are stored in-memory or on-disk via `.xai.npz` files. Each bundle contains one method and one or more frames. Each frame contains its target, timestamp, divergence flag, metadata, and attribution grids organized as `attributions[input_var][level_id]`. Surface layers use `sfc`; atmospheric layers use viewer-level keys such as `z-N`, with display labels such as `850 hPa` supplied separately. As shown in Fig. 5, the saved archive contains a `meta.json` manifest and one `float32` array per frame, input variable, and layer.

Overlay bundles are stored separately as `.overlay.npz` files. They describe one physical field through variable, optional level, timestamps, unit, color map, visibility, and one `float32` grid per frame. Remote execution uses JSON for requests and status updates, while completed attribution or overlay payloads are returned as `msgpack` and immediately reconstructed.

F Remote Execution Data Flow

A call such as `run_ig(target, input=[...], remote=...)` supplies the target specification, method, and input variables. Omitting `remote` runs in the current GPU process; supplying a URL sends the same request to a listener, commonly reached through an SSH tunnel on HPC systems. The listener can run work in-process, forward to a persistent GPU worker, or submit one SLURM job per request, as summarized in Fig. 6.

The HTTP API is intentionally narrow. `POST /run` submits one attribution target, `POST /run_batch` submits multiple target timestamps, and `POST /overlay` requests raw ERA5 overlay fields, each of which returns a `job_id`. The client then polls `GET /jobs/{id}` for queue state, progress, ETA, log tail, and the result URL. When the job is complete, `GET /jobs/{id}/result` returns an attribution or overlay result bundle, and `GET /health` reports backend mode and queue state.

After dispatch, the adapter interprets the target timestamp as Aurora's second input time step t_1 and loads the matching case with $t_0 = t_1 - 6$ hours. The requested inputs are split into atmospheric and surface tensors, and the target resolver turns the target specification into a differentiable scalar over Aurora's prediction: a nearest-cell point value or a box mean at the requested surface or pressure level.

Method runners return attribution maps keyed by input variable. Saliency and Integrated Gradients split atmospheric output into per-level grids and store surface output under `sfc`. RISE and ViT-CX perturb atmospheric columns horizontally and therefore store one level-collapsed atmospheric map. The adapter wraps the maps, labels, divergence flags, runtime metadata, and optional target scores into an attribution result bundle. Batch time frames and rollouts add further frames.

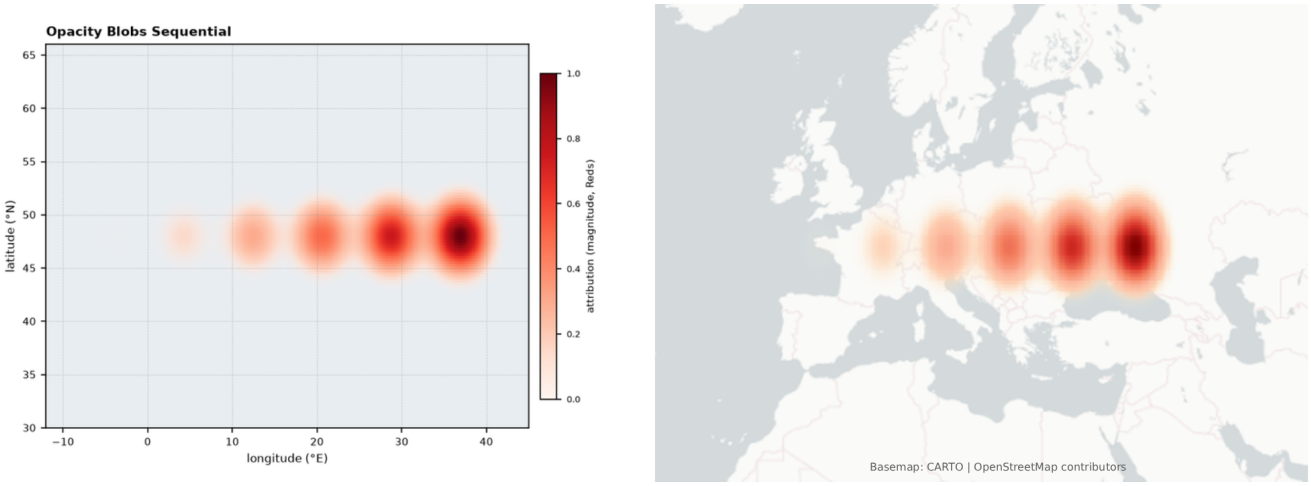


Fig. 4: Example output from the viewer visual parity suite. The left panel is the independent Matplotlib reference for the sequential opacity-blob case, and the right panel is a capture of the GeoXplain viewer under the matching launch state. This case checks the sequential color map and opacity transfer: weak blobs remain faint, while stronger blobs saturate toward deep red over the map background.

RESULT BUNDLES

my_result.xia.npz · attribution maps

```
├─ meta.json          # the contract (below)
├─ f0__t__z-0.npy     # frame 0 · ∂/∂t · 1000 hPa
├─ f0__t__z-1.npy     # frame 0 · ∂/∂t ·  925 hPa
├─ …                       one .npy per (var, level)
├─ f0__q__z-12.npy    # frame 0 · ∂/∂q ·   50 hPa
└─ f0__t2__sfc.npy    # surface var
```

`meta.json` · one method, many frames

`method`	attribution method id, "saliency"
`method_label`	display name (optional)
`layer_labels`	level key → "1000 hPa", …
`meta`	bundle-level diagnostics
`frames[ ]`	one entry per time step:
· timestamp	ISO-8601 of the frame
· target	what was explained: var, level, mode, lat, lon, size
· diverging	true if values are signed (±)
· meta	runtime_s, host, target_score
· attributions_index	var → [level keys] · lists the .npy members

my_overlay.overlay.npz · weather-field context

```
├─ meta.json          # the contract (below)
├─ f0.npy             # frame 0 weather field
├─ f1.npy             # frame 1 weather field
├─ …                       one .npy per frame
├─ lat.npy            # optional coord vector
└─ lon.npy            # optional coord vector
```

`meta.json` · one variable, many frames

`variable`	ERA5 field name, "q"
`level`	pressure in hPa, or null = surface
`label`	display name, "Specific humidity"
`unit`	value units, "kg/kg"
`colormap`	render ramp, "viridis"
`visible`	layer shown on load
`overlay_offset_hours`	time shift vs. frame stamp 0, -6, -12
`time_label`	free-text note by the offset (v3, optional)
`meta`	free-form diagnostics
`frames[ ]`	timestamp + member → f{i}.npy

All arrays are **float32 (721, 1440)** on the ERA5 0.25° grid.

Fig. 5: GeoXplain result bundles. Attribution bundles store a manifest plus one array per frame, input variable, and layer. Overlay bundles store one weather field over time with display metadata and optional coordinate vectors.

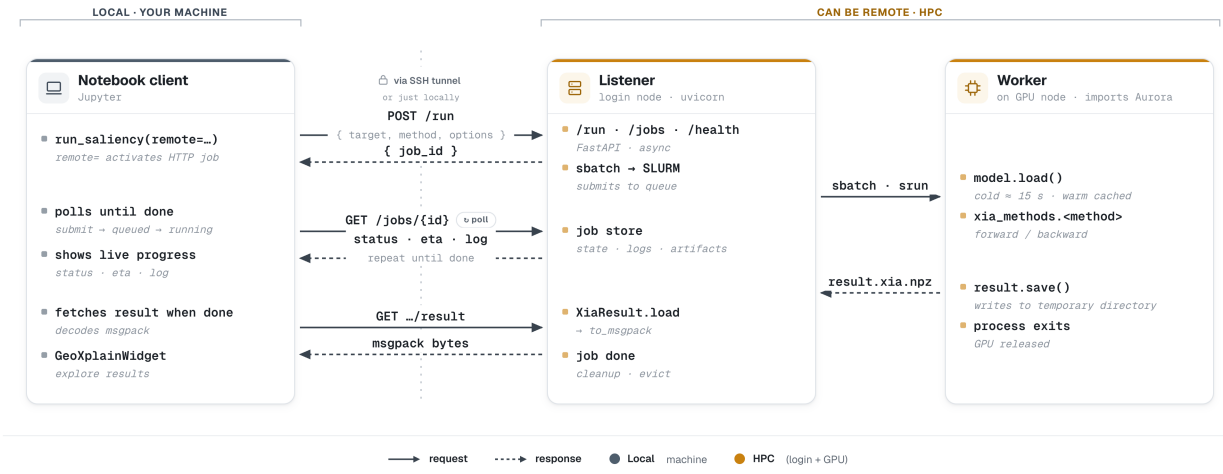


Fig. 6: Remote execution in the Aurora adapter. A notebook client submits an HTTP job, polls the listener for progress and logs, and fetches packed result bytes. On HPC systems, the listener can hand the computation to a GPU worker through SLURM while preserving the same client-side result object.